

Aufgabe 1: Schmucknachrichten

Teilnahme-ID: 76626

Bearbeiter/-in dieser Aufgabe:
Quirin Klemm

27. April 2025

Inhaltsverzeichnis

1	Lösungsidee	2
1.1	Perlen mit gleichen Durchmessern	2
1.2	Perlen mit unterschiedlichen Durchmessern	5
2	Umsetzung	6
2.1	Erstellung des Baums	6
2.2	Verbesserung des Baums	7
3	Laufzeit und Diskussion	8
3.1	Analyse des Algorithmus	8
3.2	Stand der Forschung	9
3.3	Fazit	9
4	Beispiele	10
4.1	schmuck0.txt	10
4.2	schmuck00.txt	10
4.3	schmuck01.txt	10
4.4	schmuck1.txt	10
4.5	schmuck2.txt	10
4.6	schmuck3.txt	10
4.7	schmuck4.txt	10
4.8	schmuck5.txt	10
4.9	schmuck6.txt	11
4.10	schmuck7.txt	11
4.11	schmuck8.txt	11
4.12	schmuck9.txt	12
5	Quellcode	14
5.1	TreeNode Klasse	14
5.2	main.py	15
6	Quellen	16

1 Lösungsidee

Laut Aufgabenstellung besteht eine Nachricht aus mehreren unterschiedlichen Buchstaben. Für jeden Buchstaben ist ein Codewort zu finden, wobei insgesamt ein präfixfreier Code entstehen muss. Dabei soll die Gesamtlänge der codierten Nachricht möglichst gering sein. Daher wird ein präfixfreier Code gesucht, bei dem häufig vorkommende Buchstaben kurze Codewörter erhalten, während seltenere Buchstaben längere Codewörter zugewiesen bekommen.

1.1 Perlen mit gleichen Durchmessern

Im Allgemeinen gilt, solange alle Perlen im Durchmesser gleich sind:

$$c_i = p_i \cdot n_i \quad (1)$$

$$l = \sum_i c_i \quad (2)$$

- c_i die Kosten des Buchstaben i
- p_i die Häufigkeit des Buchstabens i in der Nachricht
- n_i die Länge des Codewortes für den Buchstaben i (Anzahl der Perlen)
- l die Länge der codierten Nachricht ist

Die Gesamtlänge der Nachricht entspricht der Summe aller Kosten jedes einzelnen Buchstaben. Die Kosten beschreiben dabei, um wie viel die codierte Nachricht wegen dem Buchstaben länger wird. Um einen präfixfreien Code zu erzeugen, habe ich mich entschieden einen Baum zu benutzen. Da durch diesen sicher gestellt werden kann, dass kein Codewort der Anfang eines anderen Codeworts ist. Nach meiner Definition ist jedes Blatt des Baums ein Codewort und jede Kante zum Blatt ein Teil des Codeworts. Der Baum hat mindestens so viele Blätter wie es verschiedene Buchstaben gibt. Jeder innere Knoten hat dabei maximal so viele Kinder, wie es verschiedene Perlen gibt. Die Ebene des Blatts entspricht der Länge des Codeworts, also n .

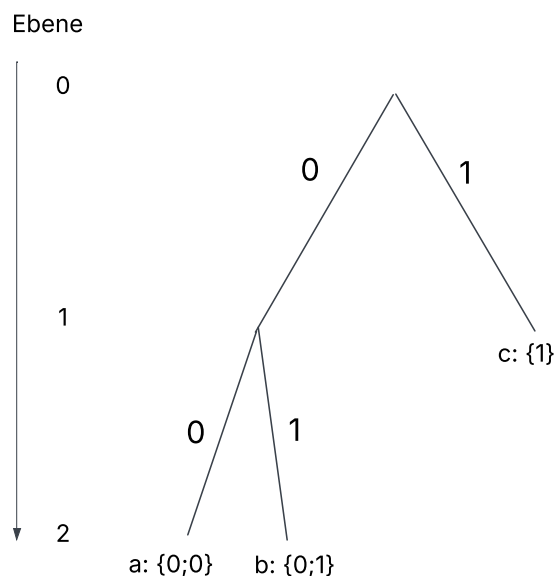


Abbildung 1: simple tree

Um einen möglichst kleinen präfixfreien Code zu erstellen, muss der Buchstabe, der am häufigsten vorkommt, ganz oben eingeordnet werden und die nächsten Buchstaben dann in absteigender Reihenfolge. In Abbildung 1 hätte c die höchste Häufigkeit und danach kommt a und b . Ob der Buchstabe a oder b die höhere Häufigkeit besitzt, ist nicht relevant, da beide auf der untersten Ebene sind.

Daher ergibt sich $p_1 \geq p_2 \geq p_3 \dots \geq p_i$ und $n_1 \leq n_2 \leq n_3 \dots \leq n_i$

Zu finden ist also ein Baum, der basierend auf den Häufigkeiten der Buchstaben, seine Blätter so auf seine Ebenen verteilt, dass die Länge der codierten Nachricht (Gleichung (2)) möglichst gering ist. Jeder innere Knoten muss dabei mindestens zwei Kinder haben, da ein innerer Knoten mit nur einem Kind die Äste des Baums verlängert (höheres n) ohne einen Vorteile zu bringen.

Zu Beginn des Algorithmus wird ein balancierter Baum erstellt, der so viele Blätter wie Buchstaben hat (Abbildung 2). Jedes Blatt wird von links nach rechts in aufsteigender Reihenfolge der Häufigkeit, mit einem Buchstaben belegt, und die Gesamtkosten des Baums werden gemäß der Formel (2) berechnet. Dabei ergeben sich deutlich höhere Kosten für die rechten Buchstaben im Baum im Vergleich zu den linken, da alle Blätter auf derselben Ebene sind, jedoch die Häufigkeit der Buchstaben auf der rechten Seite größer ist. Um dies auszugleichen, wird der Buchstabe mit den höchsten Kosten eine Ebene nach oben verschoben, indem der Elternknoten des Blattes selber zum Blatt wird und alle Kinder verliert. Dadurch verliert der Baum aber zu viele Kinder, weshalb er erweitert werden muss. Für die Erweiterung untersucht der Algorithmus, an welche Knoten neue Blätter angehängt werden können. Er überprüft die Blätter nach ihrer Ebene, d.h. nach ihrem Wert für n , und fügt das mit dem kleinsten Wert hinzu. Das nach oben verschobenene Blatt erhält den Zusatz *protected*. Das verhindert, dass es selbst weitere Kinder bekommen kann. Nach jeder Iteration werden den Blättern wie zu Beginn von links nach rechts Buchstaben aufsteigend hinzugefügt. Das stellt sicher, dass immer die äußerst linken Blätter (d.h. die mit dem höchsten n -Wert), die Buchstaben mit den geringsten Häufigkeiten kriegen und rechts die höchsten. Dadurch wird ein Zyklus vermieden, bei dem ein Blatt zunächst nach oben verschoben und anschließend wieder nach unten erweitert würde.

Das Verschieben der Blätter nach oben, sowie das Neueinfügen der Blätter unten, gilt als ein Optimierungsschritt, welcher im Optimierungsprozess wiederholt wird. (Abbildung 4) Dieser Vorgang wird so lange fortgesetzt, bis keine neuen Blätter mehr eingefügt werden können, da alle Blätter als *protected* gelten (Abbildung 5) - also keine weiteren Optimierungen mehr möglich sind. Das ist der Fall kurz vor einem maximal ausgearteten Baum. Der Algorithmus geht also von einem Maximum (maximal ausbalanciert) zum anderen Maximum (maximal ausgeartet). Am Ende wird der Baum mit den geringsten Gesamtkosten ausgegeben.

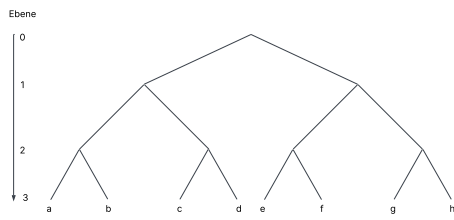


Abbildung 2: Balancierter Baum

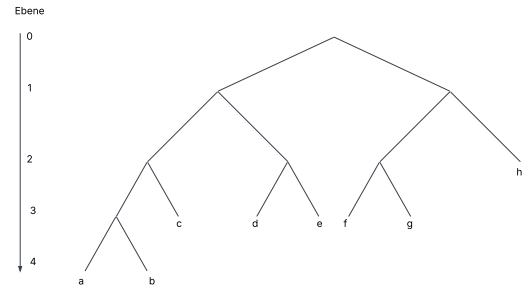


Abbildung 3: Transformation 1

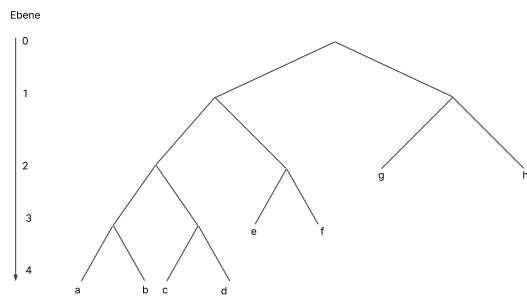


Abbildung 4: Transformation 2

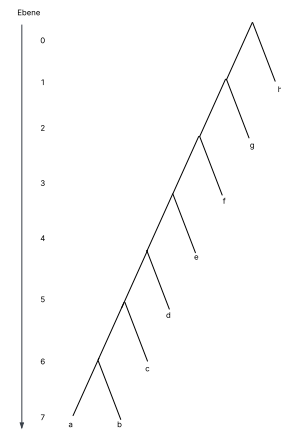


Abbildung 5: Ausgearteter Baum

1.2 Perlen mit unterschiedlichen Durchmessern

Das Problem bei unterschiedlichen Durchmessern besteht darin, dass Blätter auf derselben Ebene unterschiedliche Kosten für die Gesamtlänge der Nachricht haben können. Daher sind die Kosten nicht mehr der Wert der Ebene und damit die Anzahl an Perlen. Das hat zu Folge, dass die Gleichung (1) nicht mehr stimmt. Stattdessen muss der Durchmesser jeder Perle im Codewort einzeln angeschaut werden. Daher ergibt sich

$$c_i = p_i \cdot \sum_n cp_n \quad (3)$$

- cp_n die Kosten/Durchmesser der Perle n

Stellt man nun Bäume nicht nach ihrer Ebene, sondern nach ihren Kosten dar, erhält man einen *lopsided tree* [1].

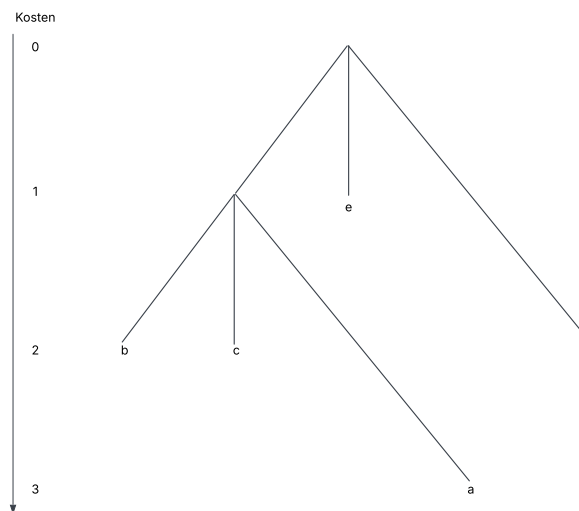


Abbildung 6: lopsided tree: Perlendurchmesser: (1; 1; 2)

Auch hier gilt: Jeder innere Knoten muss mindestens zwei Kinder haben. Bei dieser Art von Bäumen hat jedoch nicht der äußerst rechte Knoten die geringsten Kosten, sondern der oberste. Der Algorithmus bleibt unverändert: Zunächst wird ein Baum erstellt, in dem alle Blätter ungefähr die gleichen Kosten haben. Anschließend wird der Buchstabe mit den höchsten Kosten nach oben verschoben, während der Baum unten erweitert wird. Dies geschieht solange, bis kein Blatt mehr eingefügt werden kann und der beste Baum wird ausgegeben.

Wenn die Durchmesser alle gleich groß sind, gilt:

$$p_i \cdot n_i = p_i \cdot \sum_n cp_n \quad (4)$$

daher kann der zweite Algorithmus sowohl für gleiche als auch unterschiedliche Durchmesser benutzt werden.

2 Umsetzung

Die Lösung wurde in Python3 geschrieben. Die Eingabetexte werden eingelesen. Die Durchmesser der Perlen werden in einer Liste gespeichert und der Text in einem string. Aus dem Text werden anschließend zwei Listen erstellt: Die erste enthält die Häufigkeiten der Buchstaben in absteigender Reihenfolge. Die zweite enthält die Buchstaben entsprechend dieser Reihenfolge. Sie sehen für Beispielaufgabe *schmuck0* so aus:

1. [6, 3, 3, 2, 2, 1]

2. ['d', 'a', 's', 'f', 'g', 'h']

Um später den Buchstaben die Häufigkeit wieder richtig zuzuordnen.

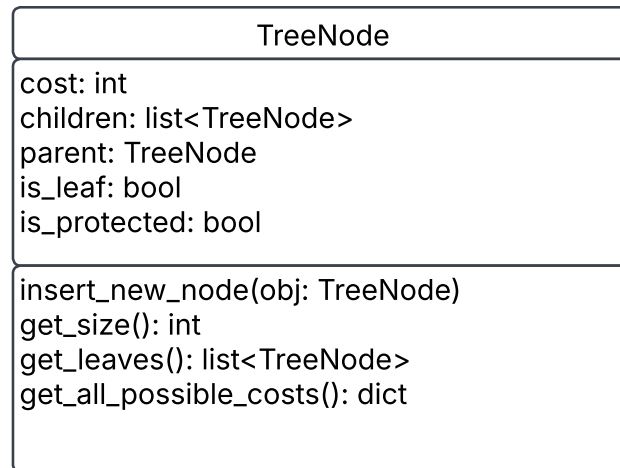


Abbildung 7: TreeNode Klasse

2.1 Erstellung des Baums

Zu Beginn wird ein blanchierter Baum über die Methode *create_tree()* erstellt.

```
1 def create_tree(root: TreeNode) -> TreeNode:
```

Listing 1: Methode *create_tree*

Die Methode *create_tree* bekommt jedes potenziell neue Blatt im Baum über den Aufruf *get_all_possible_costs()* und das Blatt mit den geringsten Kosten wird dem Baum über den Aufruf *insert_new_node(obj)* hinzugefügt. Falls der Elternknoten des neuen Blatts selber noch ein Blatt ist, entsprechen die Kosten des neuen Blatts den kombinierten Kosten für die ersten beiden Blätter. Das liegt daran, dass jeder innere Knoten mindesten zwei Kinder haben muss –daher stellen diese Kosten die effektiv neu entstehenden Kosten für den Baum dar. Das geschieht solange, bis der Baum genau so viele Blätter hat, wie es Buchstaben gibt. Seine Anzahl an Blättern erhält er über die Methode *get_size()*.

```
1 def get_all_possible_costs(self) -> dict:
```

Listing 2: Methode *get_all_possible_costs*

Die Methode *get_all_possible_costs* prüft, ob das *TreeNode*-Objekt noch weitere Kinder aufnehmen kann und nicht protected ist. Falls ja, fügt es sich selbst sowie die Kosten des potenziellen neuen Kindes einem Dictionary hinzu. Anschließend wird die Methode rekursiv für alle bestehenden Kinder aufgerufen, so dass auch deren mögliche Erweiterungen im Dictionary erfasst werden.

```
1 def insert_new_node(self, obj: TreeNode):
```

Listing 3: Methode `insert_new_node`

Die Methode `insert_new_node` prüft, ob das aktuelle Objekt selbst das gesuchte `TreeNode` Objekt ist. Falls ja, fügt es ein neues Kind hinzu. Wenn vorher noch keine Kinder vorhanden sind, werden zwei Kinder hinzugefügt. Ansonsten wird die Methode rekursiv auf alle bestehenden Kinder angewandt.

```
1 def get_size(self) -> int:
```

Listing 4: Methode `get_size`

Die Methode `get_size` gibt den Wert 1 zurück falls das Objekt ein Blatt ist. Ansonsten wird die Methode rekursiv bei den Kinder aufgerufen und die Summe zurückgegeben

2.2 Verbesserung des Baums

Sobald ein balancierter Baum erstellt wurde, muss dieser optimiert werden. Dies geschieht über die Methode `find_best_tree`.

```
1 def find_best_tree(root: TreeNode) -> TreeNode:
```

Listing 5: Methode `find_best_tree`

Die Methode `find_best_tree` versucht, den übergebenen Baum so lang zu optimieren, bis alle Blätter als *protected* markiert sind und keine Blätter mehr hinzugefügt werden können. Dazu wird der Optimierungsschritt über den Aufruf `improve_tree(root)` wiederholt und jeweils mit der Methode `calculate_tree(root)` die neuen Kosten des Baums berechnet. Sind die neuen Kosten ein neues Minimum wird der aktuelle Baum als bester Zwischenstand gespeichert. Können keine neuen Blätter dem Baum hinzugefügt werden und es tritt ein Fehler auf, wird der bis dahin beste gefundene Baum zurückgegeben.

```
1 def improve_tree(root: TreeNode) -> TreeNode:
```

Listing 6: Methode `improve_tree`

Zu Beginn holt sich die Methode `improve_tree` alle Blätter im Baum über den Aufruf `get_leaves()`. Diese Liste wird dann aufsteigend sortiert und die Kosten für jedes Element können mit der Gleichung (3) berechnet werden. Der Elternknoten des Blatts mit den höchsten Kosten wird daraufhin selbst zu einem Blatt. Dafür werden die Attribute des Elternknoten

```
1 is_leaf = True
  is_protected = True
3 children = []
```

gesetzt. Um die fehlenden Blätter wieder hinzuzufügen wird noch die Methode 1 (`create_tree`) aufgerufen.

```
1 def calculate_tree(root: TreeNode) -> TreeNode:
```

Listing 7: Methode `calculate_tree`

Die Methode `calculate_tree` berechnet die gesamte Kosten des Baums. Dafür holt sie sich auch über den Aufruf `get_leaves()` alle Blätter im Baum. Die Liste an Blättern wird dann nach den Kosten aufsteigend sortiert und jedes Element wird wie in der Gleichung (3) berechnet und zusammenaddiert.

3 Laufzeit und Diskussion

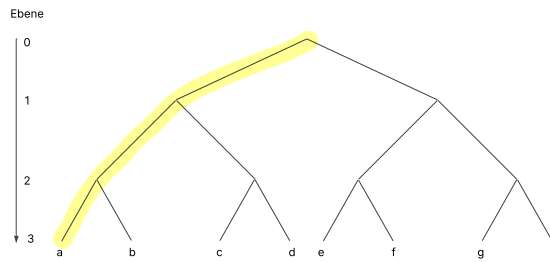


Abbildung 8: Balancierter Baum, Makierung der richtigen Knoten

3.1 Analyse des Algorithmus

Der Algorithmus endet, wenn keine Knoten mehr eingefügt werden können. Dies ist spätestens gegeben, wenn der Baum maximal ausgeartet ist (Abbildung 5). Allgemein kann jeder Knoten maximal $r - 1$ Mal verschoben werden, bis er komplett auf der linken Seite ist. Dabei ist r die Anzahl an verschiedenen Perlen. Daraus folgt, dass die maximale Anzahl an Iterationen

$$I < (r - 1) \cdot \text{Anzahl falscher Knoten} \quad (5)$$

ist, bis zum maximal ausgearteten Baum. Falsche Knoten sind dabei innere Knoten, die beim ausbalancierten Baum noch nicht auf der richtigen Stelle sind, also alle bis auf die äußerst linken Knoten. (Abbildung 8)

Generell gilt, um die maximale Anzahl an inneren Knoten zu erhalten, sollte jeder Knoten möglichst wenig Kinder besitzen. Wie zuvor festgelegt, hat jeder innere Knoten mindestens zwei Kinder. Daraus folgt, dass die maximale Anzahl an inneren Knoten erreicht wird, wenn der Baum ein Binärbaum ist und kann durch

$$\text{innere Knoten} = w - 1 \quad (6)$$

berechnet werden. Dabei ist w die Anzahl an Blättern/Codewörter. Auf jeder Ebene existiert ein richtiger innere Knoten, deren Anzahl minimal mit

$$\text{richtige innere Knoten} = \log_{r-1} w \quad (7)$$

abgeschätzt werden kann.

Daher ergibt sich

$$I < (r - 1) \cdot (w - 1 - \log_{r-1} w) \quad (8)$$

Dies bedeutet eine Laufzeit von $O(r \cdot w)$

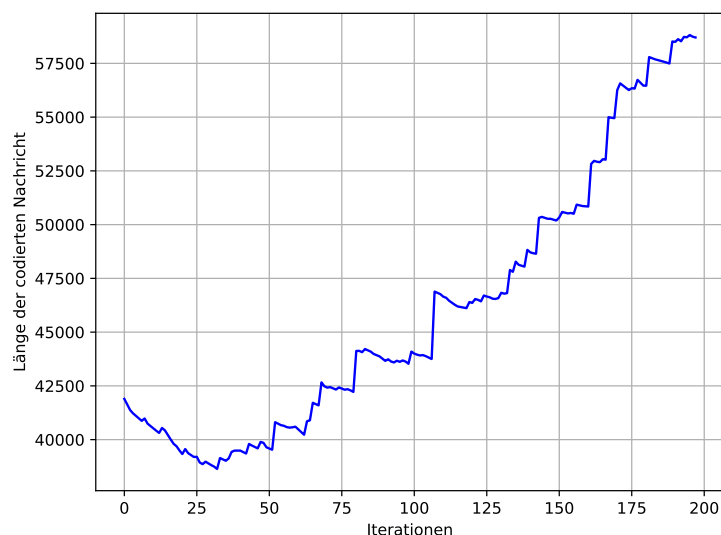


Abbildung 9: Länge der codierten Nachricht über mehrere Iterationen

Abbildung 9 zeigt wie die Länge der codierten Nachricht der Beispielaufgabe *schmuck9* über die Iterationen hinweg verändert. Das Minimum wird bereits in der ersten Hälfte der Iterationen erreicht. Dieses lässt auch bei den anderen Beispielaufgabe sehen. Daher ist es unproblematisch, dass der Algorithmus nicht bis zum maximal ausgearteten Baum iteriert. (Ein formaler Beweis dafür steht aber noch aus.)

3.2 Stand der Forschung

Die vorliegende Fragestellung ist seit den 60iger-Jahren in der theoretischen Informatik bekannt. Dabei lieferte Huffman eine Lösung für den Spezialfall, von zwei Quellsymbolen. [3]

Für unseren Fall von mehr als zwei Quellsymbolen mit unterschiedlichen Kosten, entwickelten Mordecai J. Golin und Günter Rote im Jahr 1998 das bis jetzt schnellste Verfahren, um auf eine exakte Lösung zu kommen.

”The minimum-cost prefix-free code for n words can be computed in $O(n^{C+2})$ time and $O(n^{C+1})$ space, if the costs of the letters are integers between 1 and C .”

Für die Beispielaufgabe *schmuck5.txt* hat der Algorithmus eine Laufzeit 34^8 ($n=34$; $C=6$) Iterationen. Diese Art von Algorithmus kann wegen seiner langen Laufzeit nicht für die vorgegebenen Beispielaufgabe verwendet werden. [1]

Um eine polynominelle Laufzeit zu erreichen, entwickelten Mordecai J. Golin, Claire Mathieu und Neal E. Young 2012 ein Approximationsverfahren.

Dieser Algorithmus basiert auf dem PTAS-Verfahren, wobei zuerst ein präfixfreier Code bis zur einer bestimmten Tiefe τ erstellt wird und dann die restlichen Codewörter durch ein Rundungsverfahren ergänzt werden. Dabei ist es zentral, dass die Wahrscheinlichkeitsverteilung der zu kodierenden Codewörter möglichst in großen Gruppen mit gleich großer Wahrscheinlichkeit eingeordnet werden können. In den Beispielen, vor allem in Beispiel 9 ist dies nicht gegeben, weshalb das Bestimmen des Vorbaums über 9^{24} Schritte benötigt. Somit ist dieses Verfahren für die gewählten Beispiele ebenfalls nicht verwendbar. [2]

3.3 Fazit

Mein vorgestellter Algorithmus findet nicht die beste Lösung für das gegebene Problem. Er liefert jedoch in polynomieller Laufzeit eine gute Approximation und ist daher eine gute Abwägung zwischen Laufzeit und Genauigkeit. Aus diesem Grund erfolgt durch das vorgestellte Verfahren keine Beantwortung, ob das Problem NP-hart ist. Im Gegensatz zum oben vorgestellten Verfahren kann der Algorithmus auch Fälle verarbeiten, in denen die Wahrscheinlichkeiten der Zeichen nicht gleichmäßig verteilt oder in klaren Gruppen strukturiert sind.

In praktischen Tests zeigte sich, dass die berechneten Lösungen im Durchschnitt nur etwa 10 % von der optimalen Lösung abweichen. Ein formaler Beweis dieser Beobachtung steht allerdings noch aus.

4 Beispiele

4.1 schmuck0.txt

Gesamtkosten: 114

E: [1, 0, 0] ' ' : [1, 0, 1] I: [1, 1, 0] N: [1, 1, 1] S: [0, 0, 0, 0] D: [0, 0, 0, 1] O: [0, 0, 1, 0] L: [0, 0, 1, 1] R: [0, 1, 0, 0] M: [0, 1, 0, 1] C: [0, 1, 1, 0] H: [0, 1, 1, 1]

4.2 schmuck00.txt

Gesamtkosten: 382

' ' : [0] e: [1, 1] i: [2, 0] t: [2, 1, 1] n: [2, 1, 2] s: [2, 2, 0] h: [2, 2, 1] c: [2, 2, 2] l: [2, 1, 0, 0] a: [2, 1, 0, 1] r: [1, 0, 0, 0] d: [1, 0, 0, 1] D: [1, 0, 0, 2] u: [1, 0, 1, 0] w: [1, 0, 1, 1] m: [1, 0, 1, 2] G: [1, 0, 2, 0] g: [1, 0, 2, 1] E: [1, 0, 2, 2] k: [1, 2, 0, 0] f: [1, 2, 0, 1] W: [1, 2, 0, 2] Z: [1, 2, 1, 0] P: [1, 2, 1, 1] o: [1, 2, 1, 2] ? : [1, 2, 2, 0] A: [1, 2, 2, 1] b: [1, 2, 2, 2]

4.3 schmuck01.txt

Gesamtkosten: 1165

' ' : [1] e: [2, 1] n: [2, 2] r: [2, 3] i: [2, 4] s: [3, 0] l: [3, 1] a: [3, 2] t: [3, 3] h: [3, 4] c: [4, 0] o: [4, 1] g: [4, 2] m: [4, 3] u: [4, 4] . : [0, 0, 0] d: [0, 0, 1] k: [0, 0, 2] E: [0, 0, 3] . : [0, 0, 4] D: [0, 1, 0] b: [0, 1, 1] w: [0, 1, 2] ü: [0, 1, 3] f: [0, 1, 4] S: [0, 2, 0] v: [0, 2, 1] I: [0, 2, 2] p: [0, 2, 3] O: [0, 2, 4] V: [0, 3, 0] ö: [0, 3, 1] z: [0, 3, 2] F: [0, 3, 3] B: [0, 3, 4] ä: [0, 4, 0] G: [0, 4, 1] K: [0, 4, 2] R: [0, 4, 3] M: [0, 4, 4] ß: [2, 0, 0] H: [2, 0, 1] N: [2, 0, 2] W: [2, 0, 3] ... : [2, 0, 4]

4.4 schmuck1.txt

Gesamtkosten: 194

e: [0, 2] t: [1, 0, 0] ' ' : [1, 0, 1] r: [1, 1, 0] i: [1, 1, 1] n: [1, 2] w: [2, 0] B: [2, 1] I: [0, 0, 0, 0] s: [0, 0, 0, 1] f: [0, 0, 1, 0] : [0, 0, 1, 1] b: [0, 0, 2] W: [0, 1, 0, 0] N: [0, 1, 0, 1] F: [0, 1, 1, 0] h: [0, 1, 1, 1] ü: [0, 1, 2] D: [1, 0, 2] u: [1, 1, 2] d: [2, 2] o: [0, 0, 0, 2] m: [0, 0, 1, 2] a: [0, 1, 0, 2] k: [0, 1, 1, 2]

4.5 schmuck2.txt

Gesamtkosten: 135

a: [0] ' ' : [1, 0, 0, 0, 0, 0, 0] b: [1, 0, 1] c: [1, 1, 0] d: [1, 0, 0, 1] e: [1, 0, 0, 0, 1] f: [1, 0, 0, 0, 0, 1] g: [1, 0, 0, 0, 0, 1] h: [1, 1, 1]

4.6 schmuck3.txt

Gesamtkosten: 279 a: [0, 0] b: [1] c: [0, 1] ' ' : [2, 0] d: [0, 2, 0] e: [2, 1] f: [0, 2, 1] g: [2, 2] h: [0, 2, 2]

4.7 schmuck4.txt

Gesamtkosten: 137

a: [1, 0, 0] b: [0, 0, 0, 0, 0, 0, 0, 0] c: [0, 0, 0, 1] d: [0, 0, 1, 0] e: [0, 1, 0, 0] f: [0, 0, 0, 0, 1] g: [0, 0, 0, 0, 0, 1] h: [1, 1] i: [0, 0, 0, 0, 0, 0, 1] j: [0, 1, 1] k: [1, 0, 1] l: [0, 0, 0, 0, 0, 0, 0, 1] m: [0, 0, 1, 1] n: [0, 1, 0, 1]

4.8 schmuck5.txt

Gesamtkosten: 3189

' ' : [0, 1] e: [1, 0] t: [1, 1, 0] i: [1, 1, 1] o: [2, 0] s: [2, 1] a: [0, 2] n: [1, 2] r: [3] c: [0, 0, 1, 0] d: [0, 0, 1, 1] l: [0, 0, 0, 0] m: [0, 0, 0, 1] h: [1, 1, 2] u: [2, 2] p: [0, 0, 2] f: [0, 3] b: [1, 3] y: [4] g: [0, 0, 0, 2] . : [1, 1, 3] v: [2, 3] x: [0, 0, 3] q: [0, 4] . : [1, 4] w: [5] T: [0, 0, 0, 3] F: [1, 1, 4] S: [2, 4] - : [0, 0, 4] A: [0, 5] H: [1, 5] 1: [6] 9: [0, 0, 0, 4] 5: [1, 1, 5] 2: [2, 5] z: [0, 0, 5] G: [0, 6] ' ' : [1, 6] j: [0, 0, 0, 5] . : [0, 0, 6]

4.9 schmuck6.txt

Gesamtkosten: 234

, : [0, 0, 1, 0] 文: [0, 0, 2] 馬: [0, 1, 0, 0] 。 : [0, 1, 1] 有: [0, 2, 0] 古: [1, 0, 0, 0] 英: [1, 0, 1] 雄: [1, 1, 0] 未: [1, 2] 遇: [2, 0, 0] 時: [2, 1] 都: [0, 0, 0, 1, 0] 無: [0, 0, 0, 0, 0, 0] 大: [0, 0, 0, 0, 1] 志: [0, 0, 0, 2] 非: [0, 0, 1, 1] 止: [0, 1, 0, 1] 鄧: [0, 1, 2] 禹: [0, 2, 1] 希: [1, 0, 0, 1] 學: [1, 0, 2] 、 : [1, 1, 1] 武: [2, 0, 1] 望: [2, 2] 督: [0, 0, 0, 1, 1] 郵: [0, 0, 0, 0, 0, 1] 也: [0, 0, 0, 0, 2] 匚: [0, 0, 1, 2] 公: [0, 1, 0, 2] 妻: [0, 2, 2] 不: [1, 0, 0, 2] 肯: [1, 1, 2] 去: [2, 0, 2] 齊: [0, 0, 0, 0, 0, 2]

4.10 schmuck7.txt

Gesamtkosten: 135409

' : [0] e: [1] n: [2] i: [3] r: [4, 5] s: [4, 6] a: [5, 0] d: [5, 1] h: [5, 2] t: [5, 3] u: [5, 4] l: [5, 5] c: [5, 6] g: [6, 0] m: [6, 1] o: [6, 2] b: [6, 3] ,: [6, 4] f: [6, 5] w: [6, 6] .: [7] k: [4, 0, 0] z: [4, 0, 1] S: [4, 0, 2] ä: [4, 0, 3] ü: [4, 0, 4] ö: [4, 0, 5] v: [4, 0, 6] p: [4, 1, 0] F: [4, 1, 1] ß: [4, 1, 2] D: [4, 1, 3] E: [4, 1, 4] V: [4, 1, 5] W: [4, 1, 6] N: [4, 2, 0] K: [4, 2, 1] B: [4, 2, 2] A: [4, 2, 3] M: [4, 2, 4] H: [4, 2, 5] I: [4, 2, 6] L: [4, 3, 0] R: [4, 3, 1] G: [4, 3, 2] T: [4, 3, 3] Z: [4, 3, 4] j: [4, 3, 5] J: [4, 3, 6] -: [4, 4, 0] :: [4, 4, 1] (: [4, 4, 2]): [4, 4, 3] :: [4, 4, 4] _ : [4, 4, 5] O: [4, 4, 6] U: [4, 7] [: [5, 7]]: [6, 7] ,,: [8] “: [4, 0, 7] P: [4, 1, 7] 1: [4, 2, 7] 4: [4, 3, 7] 3: [4, 4, 7] ? : [4, 8] 2: [5, 8] !: [6, 8] 5: [9] 6: [4, 0, 8] 0: [4, 1, 8] q: [4, 2, 8] Ä: [4, 3, 8] 8: [4, 4, 8] 7: [4, 9] 9: [5, 9] Q: [6, 9] ' : [4, 0, 9] Ö: [4, 1, 9] x: [4, 2, 9] y: [4, 3, 9] C: [4, 4, 9]

4.11 schmuck8.txt

Gesamtkosten: 3358

, : [0, 0, 1] 。 : [0, 0, 3] 「 : [0, 1, 0, 0, 1] 」 : [0, 1, 0, 1, 0] 》 : [0, 1, 0, 1, 1] : : [0, 1, 0, 2] 《 : [0, 1, 0, 3] 之: [0, 1, 1, 0, 0] 格: [0, 1, 1, 0, 1] 不: [0, 1, 1, 1, 0] 有: [0, 1, 1, 1, 1] 匚: [0, 1, 1, 2] 云: [0, 1, 1, 3] 時: [0, 1, 2, 0] 其: [0, 1, 2, 1] 一: [0, 1, 3, 0] 武: [0, 1, 3, 1] 相: [0, 1, 4] 人: [0, 2, 0, 0] 詩: [0, 2, 0, 1] 風: [0, 2, 1, 0] 也: [0, 2, 1, 1] 公: [0, 2, 2] 是: [0, 2, 3] 知: [0, 3, 0, 0] 在: [0, 3, 0, 1] 誰: [0, 3, 1, 0] ; : [0, 3, 1, 1] 未: [0, 3, 2] 都: [0, 3, 3] 、 : [0, 4, 0] 馬: [0, 4, 1] 嚴: [1, 0, 0, 0, 0] 而: [1, 0, 0, 0, 1] 意: [1, 0, 0, 1, 0] 以: [1, 0, 0, 1, 1] 言: [1, 0, 0, 2] 日: [1, 0, 0, 3] 十: [1, 0, 1, 0, 0] 百: [1, 0, 1, 0, 1] 皆: [1, 0, 1, 1, 0] 作: [1, 0, 1, 1, 1] 者: [1, 0, 1, 2] 花: [1, 0, 1, 3] 天: [1, 0, 2, 0] 調: [1, 0, 2, 1] 性: [1, 0, 3, 0] 情: [1, 0, 3, 1] 律: [1, 0, 4] 骨: [1, 1, 0, 0, 0] 無: [1, 1, 0, 0, 1] 大: [1, 1, 0, 1, 0] 志: [1, 1, 0, 1, 1] 非: [1, 1, 0, 2] 禹: [1, 1, 0, 3] 文: [1, 1, 1, 0, 0] 光: [1, 1, 1, 0, 1] 與: [1, 1, 1, 1, 0] 李: [1, 1, 1, 1, 1] 通: [1, 1, 1, 2] 尤: [1, 1, 1, 3] 目: [1, 1, 2, 0] 曰: [1, 1, 2, 1] 君: [1, 1, 3, 0] 得: [1, 1, 3, 1] 韓: [1, 1, 4] 王: [1, 2, 0, 0] 後: [1, 2, 0, 1] 見: [1, 2, 1, 0] 解: [1, 2, 1, 1] 林: [1, 2, 2] 將: [1, 2, 3] 開: [1, 3, 0, 0] 看: [1, 3, 0, 1] 來: [1, 3, 1, 0] 四: [1, 3, 1, 1] 此: [1, 3, 2] 便: [1, 3, 3] 年: [1, 4, 0] 已: [1, 4, 1] 中: [2, 0, 0, 0] 出: [2, 0, 0, 1] 七: [2, 0, 1, 0] 上: [2, 0, 1, 1] 問: [2, 0, 2] 濟: [2, 0, 3] 才: [2, 1, 0, 0] 何: [2, 1, 0, 1] 滿: [2, 1, 1, 0] 自: [2, 1, 1, 1] 水: [2, 1, 2] 外: [2, 1, 3] 多: [2, 2, 0] 枝: [2, 2, 1] 繩: [2, 3, 0] 深: [2, 3, 1] ' : [2, 4] 好: [3, 0, 0, 0] 談: [3, 0, 0, 1] 趣: [3, 0, 1, 0] 三: [3, 0, 1, 1] 篇: [3, 0, 2] 同: [3, 0, 3] 乎: [3, 1, 0, 0] 吟: [3, 1, 0, 1] 古: [3, 1, 1, 0] 英: [3, 1, 1, 1] 雄: [3, 1, 2] 遇: [3, 1, 3] 止: [3, 2, 0] 鄧: [3, 2, 1] 希: [3, 3, 0] 學: [3, 3, 1] 望: [3, 4] 督: [4, 0, 0] 郵: [4, 0, 1] 匚: [4, 1, 0] 妻: [4, 1, 1] 肯: [4, 2] 去: [4, 3] 齊: [0, 0, 0, 0, 0, 0] 貧: [0, 0, 0, 0, 0, 1] 訟: [0, 0, 0, 0, 1, 0] 逋: [0, 0, 0, 0, 1, 1] 租: [0, 0, 0, 0, 2] 於: [0, 0, 0, 0, 3] 奇: [0, 0, 0, 1, 0, 0] 歸: [0, 0, 0, 1, 0, 1] 謂: [0, 0, 0, 1, 1, 0] 寧: [0, 0, 0, 1, 1, 1] 耶: [0, 0, 0, 1, 2] 窺: [0, 0, 0, 1, 3] 盼: [0, 0, 0, 2, 0] 榮: [0, 0, 0, 2, 1] 匚: [0, 0, 0, 3, 0] 小: [0, 0, 0, 3, 1] 卒: [0, 0, 0, 4] 士: [0, 0, 2, 0, 0] 封: [0, 0, 2, 0, 1] 怒: [0, 0, 2, 1, 0] 侮: [0, 0, 2, 1, 1] 己: [0, 0, 2, 2] 奮: [0, 0, 2, 3] 拳: [0, 0, 4, 0] 毆: [0, 0, 4, 1] 般: [0, 1, 0, 0, 0, 0] 鄂: [0, 1, 0, 0, 0, 1] 西: [0, 1, 0, 0, 2] 辛: [0, 1, 0, 0, 3] 丑: [0, 1, 0, 1, 2] 元: [0, 1, 0, 1, 3] 攬: [0, 1, 0, 4] 鏡: [0, 1, 1, 0, 2] 老: [0, 1, 1, 0, 3] 門: [0, 1, 1, 1, 2] 草: [0, 1, 1, 1, 3] 生: [0, 1, 1, 4] 匚: [0, 1, 2, 2] 懷: [0, 1, 2, 3] 猶: [0, 1, 3, 2] 如: [0, 1, 3, 3] 到: [0, 2, 0, 2] 可: [0, 2, 0, 3] 郎: [0, 2, 1, 2] 玩: [0, 2, 1, 3] 詞: [0, 2, 4] 若: [0, 3, 0, 2] 料: [0, 3, 0, 3] 入: [0, 3, 1, 2] 及: [0, 3, 1, 3] 省: [0, 3, 4] 經: [0, 4, 2] 略: [0, 4, 3] 金: [1, 0, 0, 0, 2] 丞: [1, 0, 0, 0, 3] 席: [1, 0, 0, 1, 2] 心: [1, 0, 0, 1, 3] 酬: [1, 0, 0, 4] 恩: [1, 0, 1, 0, 2] 客: [1, 0, 1, 0, 3] 屈: [1, 0, 1, 1, 2] 指: [1, 0, 1, 1, 3] 世: [1, 0, 1, 4] 登: [1, 0, 2, 2] 甲: [1, 0, 2, 3] 秀: [1, 0, 3, 2] 樓: [1, 0, 3, 3] 匚: [1, 1, 0, 0, 2] 句: [1, 1, 0, 0, 3] 炊: [1, 1, 0, 1, 2] 匚: [1, 1, 0, 1, 3] 卓: [1, 1, 0, 4] 午: [1, 1, 1, 0, 2] 散: [1, 1, 1, 0, 3] 輕: [1, 1, 1, 1, 2] 絲: [1, 1, 1, 1, 3] 萬: [1, 1, 1, 4] 家: [1, 1, 2, 2] 飯: [1, 1, 2, 3] 熟: [1, 1, 3, 2] 訊: [1, 1, 3, 3] 招: [1, 2, 0, 2] 火: [1, 2, 0, 3] 斜: [1, 2, 1, 2] 陽: [1, 2, 1, 3] 樹: [1, 2, 4] 鄉: [1, 3, 0, 2] 祠: [1, 3, 0, 3] 居: [1, 3, 1, 2] 然: [1, 3, 1, 3] 侯: [1, 3, 4] 命: [1, 4, 2] 氣: [1, 4, 3] 象: [2, 0, 0, 2] 迴: [2, 0, 0, 3] 匚: [2, 0, 1, 2] 張: [2, 0, 1, 3] 桐: [2, 0, 4] 城: [2, 1, 0, 2] 則: [2, 1, 0, 3] 翰: [2, 1, 1, 2] 至: [2, 1, 1, 3] 首: [2, 1, 4] 最: [2, 2, 2] 清: [2, 2, 3] 妙: [2, 3, 2] 柳: [2, 3, 3] 陰: [3, 0, 0, 2] 春: [3, 0, 0, 3] 曲: [3, 0, 1, 2] 暮: [3, 0, 1, 3] 山: [3, 0, 4] 葉: [3, 1, 0, 2] 底: [3, 1, 0, 3] 雙: [3, 1, 1, 2] 蝴: [3, 1, 1, 3] 蝶: [3, 1, 4] 先: [3, 2, 2] 臨: [3, 2, 3] 種: [3, 3, 2] 化: [3, 3, 3] 兩: [4, 0, 2] 扈: [4, 0, 3] 匚: [4, 1, 2] 憐: [4, 1, 3] 龍: [4, 4] 鐘: [0, 0, 0, 0, 0, 2] 叟: [0, 0, 0, 0, 0, 3] 騎: [0, 0, 0, 0, 1, 2] 踏: [0, 0, 0, 0, 1, 3] 冰: [0, 0, 0, 0,

4] 星: [0, 0, 0, 1, 0, 2] 和: [0, 0, 0, 1, 0, 3] 皇: [0, 0, 0, 1, 1, 2] 𠂇: [0, 0, 0, 1, 1, 3] 𠂇: [0, 0, 0, 1, 4] 九: [0, 0, 0, 2, 2] 霄: [0, 0, 0, 2, 3] 近: [0, 0, 0, 3, 2] 增: [0, 0, 0, 3, 3] 華: [0, 0, 2, 0, 2] 色: [0, 0, 2, 0, 3] 野: [0, 0, 2, 1, 2] 仗: [0, 0, 2, 1, 3] 寶: [0, 0, 2, 4] 押: [0, 0, 4, 2] 字: [0, 0, 4, 3] 𠂇: [0, 1, 0, 0, 0, 2] 寄: [0, 1, 0, 0, 0, 3] 托: [0, 1, 0, 0, 4] 𠂇: [0, 1, 0, 1, 4] 二: [0, 1, 1, 0, 4] 楊: [0, 1, 1, 1, 4] 誠: [0, 1, 2, 4] 齋: [0, 1, 3, 4] 從: [0, 2, 0, 4] 分: [0, 2, 1, 4] 低: [0, 3, 0, 4] 拙: [0, 3, 1, 4] 空: [0, 4, 4] 架: [1, 0, 0, 0, 4] 子: [1, 0, 0, 1, 4] 腔: [1, 0, 1, 0, 4] 口: [1, 0, 1, 1, 4] 易: [1, 0, 2, 4] 描: [1, 0, 3, 4] 專: [1, 1, 0, 0, 4] 寫: [1, 1, 0, 1, 4] 靈: [1, 1, 1, 0, 4] 辦: [1, 1, 1, 1, 4] 余: [1, 1, 2, 4] 愛: [1, 1, 3, 4] 須: [1, 2, 0, 4] 半: [1, 2, 1, 4] 勞: [1, 3, 0, 4] 思: [1, 3, 1, 4] 婦: [1, 4, 4] 率: [2, 0, 0, 4] 事: [2, 0, 1, 4] 今: [2, 1, 0, 4] 能: [2, 1, 1, 4] 範: [2, 2, 4] 圍: [2, 3, 4] 否: [3, 0, 0, 4] 𠂇: [3, 0, 1, 4] 皋: [3, 1, 0, 4] 歌: [3, 1, 1, 4] 國: [3, 2, 4] 雅: [3, 3, 4] 頌: [4, 0, 4] 豈: [4, 1, 4] 定: [0, 0, 0, 0, 0, 4] 哉: [0, 0, 0, 0, 1, 4] 許: [0, 0, 0, 1, 0, 4] 渾: [0, 0, 0, 1, 1, 4] 似: [0, 0, 0, 2, 4] 成: [0, 0, 0, 3, 4] 仙: [0, 0, 2, 0, 4] 里: [0, 0, 2, 1, 4] 莫: [0, 0, 4, 4] 浪: [0, 1, 0, 0, 0, 4]

4.12 schmuck9.txt

Gesamtkosten: 38635

の: [0, 0, 0, 0] た: [0, 0, 0, 1] に: [0, 0, 1, 0] い: [0, 0, 2] し: [0, 1, 0, 0] と: [0, 0, 1, 1] を: [0, 0, 3] て: [0, 2, 1] は: [0, 1, 3] 、: [0, 3, 1] な: [1, 0, 0, 0, 1, 0] 。: [1, 0, 1, 3] が: [1, 0, 2, 0, 0, 0] る: [1, 0, 2, 0, 1] で: [1, 0, 2, 1, 0] こ: [1, 0, 2, 2] か: [1, 0, 3, 0, 0] つ: [1, 0, 3, 1] ら: [1, 1, 0, 0, 0, 0, 0] う: [1, 1, 0, 0, 0, 1] れ: [1, 1, 0, 0, 1, 0] す: [1, 1, 0, 0, 2] 𠂇: [1, 1, 0, 1, 0, 0] り: [1, 1, 0, 1, 1] き: [1, 1, 0, 2, 0] ま: [1, 1, 0, 3] そ: [1, 1, 1, 0, 0, 0] ン: [1, 1, 1, 0, 1] く: [1, 1, 1, 1, 0] よ: [1, 1, 1, 2] だ: [1, 1, 2, 0, 0] あ: [1, 1, 2, 1] さ: [1, 1, 3, 0] も: [1, 2, 0, 0, 0, 0] わ: [1, 2, 0, 0, 1] ・: [1, 2, 0, 1, 0] : [1, 2, 0, 2] ー: [1, 2, 1, 0, 0] お: [1, 2, 1, 1] け: [1, 2, 2, 0] 「: [1, 2, 3] 」: [1, 3, 0, 0, 0] ル: [1, 3, 0, 1] え: [1, 3, 1, 0] ち: [1, 3, 2] ん: [2, 0, 0, 0, 0, 0, 0] ラ: [2, 0, 0, 0, 0, 1] つ: [2, 0, 0, 0, 1, 0] ス: [2, 0, 0, 0, 2] カ: [2, 0, 0, 1, 0, 0] 人: [2, 0, 0, 1, 1] イ: [2, 0, 0, 2, 0] 見: [2, 0, 0, 3] ト: [2, 0, 1, 0, 0, 0] ウ: [2, 0, 1, 0, 1] レ: [2, 0, 1, 1, 0] ど: [2, 0, 1, 2] や: [2, 0, 2, 0, 0] ジ: [2, 0, 2, 1] 大: [2, 0, 3, 0] 聖: [2, 1, 0, 0, 0, 0] 彼: [2, 1, 0, 0, 1] め: [2, 1, 0, 1, 0] 上: [2, 1, 0, 2] エ: [2, 1, 1, 0, 0] 者: [2, 1, 1, 1] セ: [2, 1, 2, 0] ア: [2, 1, 3] 思: [2, 2, 0, 0, 0] フ: [2, 2, 0, 1] 出: [2, 2, 1, 0] ば: [2, 2, 2] サ: [2, 3, 0, 0] 車: [2, 3, 1] 堂: [3, 0, 0, 0, 0, 0] 建: [3, 0, 0, 0, 1] 海: [3, 0, 0, 1, 0] 中: [3, 0, 0, 2] む: [3, 0, 1, 0, 0] 道: [3, 0, 1, 1] ヨ: [3, 0, 2, 0] 事: [3, 0, 3] 物: [3, 1, 0, 0, 0] み: [3, 1, 0, 1] チ: [3, 1, 1, 0] 言: [3, 1, 2] タ: [3, 2, 0, 0] 手: [3, 2, 1] 地: [3, 3, 0] 一: [1, 0, 1, 2, 0, 0] 口: [0, 0, 0, 2, 0, 0, 0, 0] 会: [0, 0, 0, 2, 0, 0, 1] べ: [0, 0, 0, 2, 0, 1, 0] ク: [0, 0, 0, 2, 0, 2] げ: [0, 0, 0, 2, 1, 0, 0] 部: [0, 0, 0, 2, 1, 1] マ: [0, 0, 0, 2, 2, 0] ほ: [0, 0, 0, 2, 3] オ: [0, 0, 0, 3, 0, 0, 0] ろ: [0, 0, 0, 3, 0, 1] び: [0, 0, 0, 3, 1, 0] 立: [0, 0, 0, 3, 2] 水: [0, 0, 1, 2, 0, 0, 0] 築: [0, 0, 1, 2, 0, 1] 𠂇: [0, 0, 1, 2, 1, 0] 雨: [0, 0, 1, 2, 2] 通: [0, 0, 1, 3, 0, 0] よ: [0, 0, 1, 3, 1] 入: [0, 1, 0, 1, 0, 0, 0, 0] ね: [0, 1, 0, 1, 0, 0, 1] 主: [0, 1, 0, 1, 0, 1, 0] 任: [0, 1, 0, 1, 0, 2] 司: [0, 1, 0, 1, 1, 0, 0] 場: [0, 1, 0, 1, 1, 1] 川: [0, 1, 0, 1, 2, 0] 𠂇: [0, 1, 0, 1, 3] 自: [0, 1, 0, 2, 0, 0, 0] 馬: [0, 1, 0, 2, 0, 1] 員: [0, 1, 0, 2, 1, 0] ア: [0, 1, 0, 2, 2] ド: [0, 1, 0, 3, 0, 0] ぶ: [0, 1, 0, 3, 1] 家: [0, 1, 1, 0, 0, 0, 0, 0] 合: [0, 1, 1, 0, 0, 0, 1] : [0, 1, 1, 0, 0, 1, 0] 分: [0, 1, 1, 0, 0, 2] 𠂇: [0, 1, 1, 0, 1, 0, 0] 祭: [0, 1, 1, 0, 1, 1] 𠂇: [0, 1, 1, 0, 2, 0] 内: [0, 1, 1, 0, 3] 送: [0, 1, 1, 1, 0, 0, 0] 後: [0, 1, 1, 1, 0, 1] 港: [0, 1, 1, 1, 1, 0] ぎ: [0, 1, 1, 1, 2] 市: [0, 1, 1, 2, 0, 0] 取: [0, 1, 1, 2, 1] ツ: [0, 1, 1, 3, 0] 利: [0, 1, 2, 0, 0, 0, 0] 明: [0, 1, 2, 0, 0, 1] 行: [0, 1, 2, 0, 1, 0] ご: [0, 1, 2, 0, 2] 目: [0, 1, 2, 1, 0, 0] 𠂇: [0, 1, 2, 1, 1] 降: [0, 1, 2, 2, 0] 前: [0, 1, 2, 3] ぎ: [0, 2, 0, 0, 0, 0, 0] 友: [0, 2, 0, 0, 0, 0, 1] 奏: [0, 2, 0, 0, 0, 1, 0] 重: [0, 2, 0, 0, 0, 2] 不: [0, 2, 0, 0, 1, 0, 0] 陸: [0, 2, 0, 0, 1, 1] 四: [0, 2, 0, 0, 2, 0] 得: [0, 2, 0, 0, 3] 身: [0, 2, 0, 1, 0, 0, 0] 防: [0, 2, 0, 1, 0, 1] ぐ: [0, 2, 0, 1, 1, 0] 振: [0, 2, 0, 1, 2] 方: [0, 2, 0, 2, 0, 0] 𠂇: [0, 2, 0, 2, 1] 𠂇: [0, 2, 0, 3, 0] 外: [0, 2, 2, 0, 0, 0] 交: [0, 2, 2, 0, 1] 年: [0, 2, 2, 1, 0] 運: [0, 2, 2, 2] 屋: [0, 2, 3, 0, 0] 式: [0, 2, 3, 1] 敷: [0, 3, 0, 0, 0, 0, 0, 0] 過: [0, 3, 0, 0, 0, 1] フ: [0, 3, 0, 0, 1, 0] 着: [0, 3, 0, 0, 2] 用: [0, 3, 0, 1, 0, 0] 数: [0, 3, 0, 1, 1] 𠂇: [0, 3, 0, 2, 0] 側: [0, 3, 0, 3] ほ: [0, 3, 2, 0, 0] ひ: [0, 3, 2, 1] 若: [0, 3, 3, 0] 意: [1, 0, 0, 0, 0, 0, 0, 0, 0] 紹: [1, 0, 0, 0, 0, 0, 0, 1] 介: [1, 0, 0, 0, 0, 0, 1, 0] 教: [1, 0, 0, 0, 0, 0, 2] ミ: [1, 0, 0, 0, 0, 1, 0, 0] ノ: [1, 0, 0, 0, 0, 1, 1] ガ: [1, 0, 0, 0, 0, 2, 0] 医: [1, 0, 0, 0, 0, 3] リ: [1, 0, 0, 0, 1, 1, 0] 差: [1, 0, 0, 0, 1, 2] 安: [1, 0, 0, 0, 2, 0, 0] 素: [1, 0, 0, 0, 2, 1] 塔: [1, 0, 0, 0, 3, 0] 量: [1, 0, 0, 1, 0, 0, 0, 0] 無: [1, 0, 0, 1, 0, 0, 1] 船: [1, 0, 0, 1, 0, 1, 0] ダ: [1, 0, 0, 1, 0, 2] 𠂇: [1, 0, 0, 1, 1, 0, 0] 名: [1, 0, 0, 1, 1, 1] 河: [1, 0, 0, 1, 2, 0] 他: [1, 0, 0, 1, 3] 民: [1, 0, 0, 2, 0, 0, 0] 生: [1, 0, 0, 2, 0, 1] 代: [1, 0, 0, 2, 1, 0] 考: [1, 0, 0, 2, 2] 石: [1, 0, 0, 3, 0, 0] 堤: [1, 0, 0, 3, 1] 牧: [1, 0, 1, 0, 0, 0, 0, 0] 草: [1, 0, 1, 0, 0, 0, 1] 味: [1, 0, 1, 0, 0, 1, 0] 南: [1, 0, 1, 0, 0, 2] 越: [1, 0, 1, 0, 1, 0, 0] 舞: [1, 0, 1, 0, 1, 1] 町: [1, 0, 1, 0, 2, 0] ヌ: [1, 0, 1, 0, 3] 小: [1, 0, 1, 1, 0, 0, 0] 北: [1, 0, 1, 1, 0, 1] 走: [1, 0, 1, 1, 1, 0] 復: [1, 0, 1, 1, 2] 保: [1, 0, 1, 2, 1] 古: [1, 0, 2, 0, 0, 1] 説: [1, 0, 2, 0, 2] 修: [1, 0, 2, 1, 1] 商: [1, 0, 2, 3] ズ: [1, 0, 3, 0, 1] 到: [1, 0, 3, 2] 荷: [1, 1, 0, 0, 0, 0, 1] 足: [1, 1, 0, 0, 0, 2] 音: [1, 1, 0, 0, 1, 1] パ: [1, 1, 0, 0, 3] じ: [1, 1, 0, 1, 0, 1] 𠂇: [1, 1, 0, 1, 2] ぞ: [1, 1, 0, 2, 1] 少: [1, 1, 1, 0, 0, 1] 散: [1, 1, 1, 0, 2] 巨: [1, 1, 1, 1, 1] 空: [1, 1, 1, 3] 話: [1, 1, 2, 0, 1] 格: [1, 1, 2, 2] 助: [1, 1, 3, 1] 仕: [1, 2, 0, 0, 0, 1] 示: [1, 2, 0, 0, 2] づ: [1, 2, 0, 1, 1] 指: [1, 2, 0, 3] や: [1, 2, 1, 0, 1] 特: [1, 2, 1, 2] 廊: [1, 2, 2, 1] 晴: [1, 3, 0, 0, 1] 眠: [1, 3, 0, 2] 𠂇: [1, 3, 1, 1] オ: [1, 3, 3] 元: [2, 0, 0, 0, 0, 0, 1] 呼: [2, 0, 0, 0, 0, 2] 所: [2, 0, 0, 0, 1, 1] 今: [2,

0, 0, 0, 3] 岸: [2, 0, 0, 1, 0, 1] 二: [2, 0, 0, 1, 2] 寄: [2, 0, 0, 2, 1] 隊: [2, 0, 1, 0, 0, 1] 𠂔: [2, 0, 1, 0, 2] 世: [2, 0, 1, 1, 1] 由: [2, 0, 1, 3] 砂: [2, 0, 2, 0, 1] 流: [2, 0, 2, 2] 細: [2, 0, 3, 1] 横: [2, 1, 0, 0, 0, 1] 伸: [2, 1, 0, 0, 2] 知: [2, 1, 0, 1, 1] 放: [2, 1, 0, 3] 羊: [2, 1, 1, 0, 1] 𠂔: [2, 1, 1, 2] 持: [2, 1, 2, 1] 負: [2, 2, 0, 0, 1] 美: [2, 2, 0, 2] 風: [2, 2, 1, 1] 波: [2, 2, 3] 儀: [2, 3, 0, 1] 日: [2, 3, 2] 𠂔: [3, 0, 0, 0, 0, 1] 𠂔: [3, 0, 0, 0, 2] 動: [3, 0, 0, 1, 1] 境: [3, 0, 0, 3] ズ: [3, 0, 1, 0, 1] 輪: [3, 0, 1, 2] 街: [3, 0, 2, 1] 治: [3, 1, 0, 0, 1] 都: [3, 1, 0, 2] 有: [3, 1, 1, 1] 能: [3, 1, 3] 信: [3, 2, 0, 1] 望: [3, 2, 2] 集: [3, 3, 1] 𠂔: [1, 0, 1, 2, 0, 1] 便: [0, 0, 0, 2, 0, 0, 0, 1] 社: [0, 0, 0, 2, 0, 0, 2] 支: [0, 0, 0, 2, 0, 1, 1] 工: [0, 0, 0, 2, 0, 3] 委: [0, 0, 0, 2, 1, 0, 1] 時: [0, 0, 0, 2, 1, 2] 汽: [0, 0, 0, 2, 2, 1] 午: [0, 0, 0, 3, 0, 0, 1] 三: [0, 0, 0, 3, 0, 2] 等: [0, 0, 0, 3, 1, 1] 券: [0, 0, 0, 3, 3] 賈: [0, 0, 1, 2, 0, 0, 1] 𠂔: [0, 0, 1, 2, 0, 2] 心: [0, 0, 1, 2, 1, 1] 老: [0, 0, 1, 2, 3] プ: [0, 0, 1, 3, 0, 1] ム: [0, 0, 1, 3, 2] ホ: [0, 1, 0, 1, 0, 0, 0, 1] テ: [0, 1, 0, 1, 0, 0, 2] 床: [0, 1, 0, 1, 0, 1, 1] 覆: [0, 1, 0, 1, 0, 3] 十: [0, 1, 0, 1, 1, 0, 1] 全: [0, 1, 0, 1, 1, 2] 頭: [0, 1, 0, 1, 2, 1] 面: [0, 1, 0, 2, 0, 0, 1] 感: [0, 1, 0, 2, 0, 2] ビ: [0, 1, 0, 2, 1, 1] ヴ: [0, 1, 0, 2, 3] 進: [0, 1, 0, 3, 0, 1] 根: [0, 1, 0, 3, 2] 神: [0, 1, 1, 0, 0, 0, 0, 1] ポ: [0, 1, 1, 0, 0, 0, 2] 深: [0, 1, 1, 0, 0, 1, 1] 套: [0, 1, 1, 0, 0, 3] 暗: [0, 1, 1, 0, 1, 0, 1] 職: [0, 1, 1, 0, 1, 2] 直: [0, 1, 1, 0, 2, 1] 多: [0, 1, 1, 1, 0, 0, 1] 以: [0, 1, 1, 1, 0, 2] 𠂔: [0, 1, 1, 1, 1, 1] 参: [0, 1, 1, 1, 3] キ: [0, 1, 1, 2, 0, 1] 尊: [0, 1, 1, 2, 2] 区: [0, 1, 1, 3, 1] 下: [0, 1, 2, 0, 0, 0, 1] シ: [0, 1, 2, 0, 0, 2] 𠂔: [0, 1, 2, 0, 1, 1] ぬ: [0, 1, 2, 0, 3] 度: [0, 1, 2, 1, 0, 1] 𠂔: [0, 1, 2, 1, 2] 達: [0, 1, 2, 2, 1] 何: [0, 2, 0, 0, 0, 0, 0, 1] 役: [0, 2, 0, 0, 0, 0, 2] 濡: [0, 2, 0, 0, 0, 1, 1] 𠂔: [0, 2, 0, 0, 0, 3] 注: [0, 2, 0, 0, 1, 0, 1] べ: [0, 2, 0, 0, 1, 2] 丈: [0, 2, 0, 0, 2, 1] 𠂔: [0, 2, 0, 1, 0, 0, 1] 積: [0, 2, 0, 1, 0, 2] 山: [0, 2, 0, 1, 1, 1] 英: [0, 2, 0, 1, 3] 国: [0, 2, 0, 2, 0, 1] 測: [0, 2, 0, 2, 2] 制: [0, 2, 0, 3, 1] 作: [0, 2, 2, 0, 0, 1] 𠂔: [0, 2, 2, 0, 2] 敵: [0, 2, 2, 1, 1] 艦: [0, 2, 2, 3] 𠂔: [0, 2, 3, 0, 1] 六: [0, 2, 3, 2] 紀: [0, 3, 0, 0, 0, 0, 1] 攻: [0, 3, 0, 0, 0, 2] 迎: [0, 3, 0, 0, 1, 1] 緒: [0, 3, 0, 0, 3] 𠂔: [0, 3, 0, 1, 0, 1] 史: [0, 3, 0, 1, 2] 輝: [0, 3, 0, 2, 1] 残: [0, 3, 2, 0, 1] 域: [0, 3, 2, 2] 沈: [0, 3, 3, 1] 泥: [1, 0, 0, 0, 0, 0, 0, 0, 1] ふ: [1, 0, 0, 0, 0, 0, 0, 2] 州: [1, 0, 0, 0, 0, 0, 1, 1] 貿: [1, 0, 0, 0, 0, 0, 3] 易: [1, 0, 0, 0, 0, 1, 0, 1] 探: [1, 0, 0, 0, 0, 1, 2] へ: [1, 0, 0, 0, 0, 2, 1] 𠂔: [1, 0, 0, 0, 1, 1, 1] 縮: [1, 0, 0, 0, 1, 3] 𠂔: [1, 0, 0, 0, 2, 0, 1] 貌: [1, 0, 0, 0, 2, 2] 類: [1, 0, 0, 0, 3, 1] 計: [1, 0, 0, 1, 0, 0, 0, 1] 𠂔: [1, 0, 0, 1, 0, 0, 2] 𠂔: [1, 0, 0, 1, 0, 1, 1] 埋: [1, 0, 0, 1, 0, 3] 償: [1, 0, 0, 1, 1, 0, 1] 真: [1, 0, 0, 1, 1, 2] 路: [1, 0, 0, 1, 2, 1] 造: [1, 0, 0, 2, 0, 0, 1] 低: [1, 0, 0, 2, 0, 2] 峽: [1, 0, 0, 2, 1, 1] プ: [1, 0, 0, 2, 3] 産: [1, 0, 0, 3, 0, 1] 抵: [1, 0, 0, 3, 2] 抗: [1, 0, 1, 0, 0, 0, 0, 0, 1] 渡: [1, 0, 1, 0, 0, 0, 2] 東: [1, 0, 1, 0, 0, 1, 1] 潮: [1, 0, 1, 0, 0, 3] 共: [1, 0, 1, 0, 1, 0, 1] 忘: [1, 0, 1, 0, 1, 2] 昔: [1, 0, 1, 0, 2, 1] 拘: [1, 0, 1, 1, 0, 0, 1] 束: [1, 0, 1, 1, 0, 2] 断: [1, 0, 1, 1, 1, 1] 切: [1, 0, 1, 1, 3] 暴: [1, 0, 1, 2, 2] 階: [1, 0, 2, 0, 0, 2] 眺: [1, 0, 2, 0, 3] 誰: [1, 0, 2, 1, 2] 再: [1, 0, 3, 0, 2] 移: [1, 0, 3, 3] 浸: [1, 1, 0, 0, 0, 0, 2] 湖: [1, 1, 0, 0, 0, 3] 界: [1, 1, 0, 0, 1, 2] 幅: [1, 1, 0, 1, 0, 2] ザ: [1, 1, 0, 1, 3] 干: [1, 1, 0, 2, 2] 舍: [1, 1, 1, 0, 0, 2] 七: [1, 1, 1, 0, 3] 長: [1, 1, 1, 1, 2] 鄙: [1, 1, 2, 0, 2] 往: [1, 1, 2, 3] 選: [1, 1, 3, 2] 議: [1, 2, 0, 0, 0, 2] ゼ: [1, 2, 0, 0, 3] 体: [1, 2, 0, 1, 2] 表: [1, 2, 1, 0, 2] 正: [1, 2, 1, 3] 組: [1, 2, 2, 2] 織: [1, 3, 0, 0, 2] 改: [1, 3, 0, 3] 善: [1, 3, 1, 2] 必: [2, 0, 0, 0, 0, 0, 0, 2] 要: [2, 0, 0, 0, 0, 3] 設: [2, 0, 0, 0, 1, 2] 性: [2, 0, 0, 1, 0, 2] 去: [2, 0, 0, 1, 3] 比: [2, 0, 0, 2, 2] 粗: [2, 0, 1, 0, 0, 2] 末: [2, 0, 1, 0, 3] 𠂔: [2, 0, 1, 1, 2] 当: [2, 0, 2, 0, 2] 珍: [2, 0, 2, 3] 消: [2, 0, 3, 2] 𠂔: [2, 1, 0, 0, 0, 2] 激: [2, 1, 0, 0, 3] ロ: [2, 1, 0, 1, 2] 旅: [2, 1, 1, 0, 2] 費: [2, 1, 1, 3] 節: [2, 1, 2, 2] 約: [2, 2, 0, 0, 2] 𠂔: [2, 2, 0, 3] 威: [2, 2, 1, 2] 苦: [2, 3, 0, 2] 𠂔: [2, 3, 3] 終: [3, 0, 0, 0, 0, 2] 養: [3, 0, 0, 0, 3] 院: [3, 0, 0, 1, 2] 配: [3, 0, 1, 0, 2] 属: [3, 0, 1, 3] ズ: [3, 0, 2, 2] 族: [3, 1, 0, 0, 2] 喜: [3, 1, 0, 3] 好: [3, 1, 1, 2] 客: [3, 2, 0, 2] 余: [3, 2, 3] 裕: [3, 3, 2] お: [0, 0, 0, 2, 0, 0, 0, 0, 2] 藁: [0, 0, 0, 2, 0, 0, 3] 突: [0, 0, 0, 2, 0, 1, 2] 間: [0, 0, 0, 2, 1, 0, 2] 耐: [0, 0, 0, 2, 1, 3] 完: [0, 0, 0, 2, 2, 2] 𠂔: [0, 0, 0, 3, 0, 0, 2] 角: [0, 0, 0, 3, 0, 3] 聳: [0, 0, 0, 3, 1, 2] セ: [0, 0, 1, 2, 0, 0, 2] 嘆: [0, 0, 1, 2, 0, 3] 声: [0, 0, 1, 2, 1, 2] 抑: [0, 0, 1, 3, 0, 2] 𠂔: [0, 0, 1, 3, 3] 追: [0, 1, 0, 1, 0, 0, 0, 2] 𠂔: [0, 1, 0, 1, 0, 0, 3] 緑: [0, 1, 0, 1, 0, 1, 2] 幾: [0, 1, 0, 1, 1, 0, 2] デ: [0, 1, 0, 1, 1, 3] イ: [0, 1, 0, 1, 2, 2] ズ: [0, 1, 0, 2, 0, 0, 2] 待: [0, 1, 0, 2, 0, 3] 沛: [0, 1, 0, 2, 1, 2] 然: [0, 1, 0, 3, 0, 2] 𠂔: [0, 1, 0, 3, 3] 霧: [0, 1, 1, 0, 0, 0, 0, 2] 𠂔: [0, 1, 1, 0, 0, 0, 3] 蒸: [0, 1, 1, 0, 0, 1, 2] エ: [0, 1, 1, 0, 1, 0, 2] 被: [0, 1, 1, 0, 1, 3] 台: [0, 1, 1, 0, 2, 2] 紗: [0, 1, 1, 1, 0, 0, 2] 幕: [0, 1, 1, 1, 0, 3] 郭: [0, 1, 1, 1, 1, 2] 浮: [0, 1, 1, 2, 0, 2] 想: [0, 1, 1, 2, 3] 像: [0, 1, 1, 3, 2] 描: [0, 1, 2, 0, 0, 0, 2] 姿: [0, 1, 2, 0, 0, 3] 遙: [0, 1, 2, 0, 1, 2] 秘: [0, 1, 2, 1, 0, 2] 的: [0, 1, 2, 1, 3] 𠂔: [0, 1, 2, 2, 2] 門: [0, 2, 0, 0, 0, 0, 0, 2] 停: [0, 2, 0, 0, 0, 0, 3] 𠂔: [0, 2, 0, 0, 0, 1, 2] 板: [0, 2, 0, 0, 1, 0, 2] 御: [0, 2, 0, 0, 1, 3] 開: [0, 2, 0, 0, 2, 2] 届: [0, 2, 0, 1, 0, 0, 2] 帽: [0, 2, 0, 1, 0, 3] 子: [0, 2, 0, 1, 1, 2] 襟: [0, 2, 0, 2, 0, 2] 飛: [0, 2, 0, 2, 3] 墓: [0, 2, 0, 3, 2] 急: [0, 2, 2, 0, 0, 2] 服: [0, 2, 2, 0, 3] 撥: [0, 2, 2, 1, 2] 扉: [0, 2, 3, 0, 2] 𠂔: [0, 2, 3, 3] 革: [0, 3, 0, 0, 0, 0, 2] 帳: [0, 3, 0, 0, 0, 3] 《: [0, 3, 0, 0, 1, 2] 》: [0, 3, 0, 1, 0, 2] 押: [0, 3, 0, 1, 3] 雲: [0, 3, 0, 2, 2] 閉: [0, 3, 2, 0, 2] 薄: [0, 3, 2, 3] 歌: [0, 3, 3, 2] 席: [1, 0, 0, 0, 0, 0, 0, 0, 2] 男: [1, 0, 0, 0, 0, 0, 0, 3] 返: [1, 0, 0, 0, 0, 0, 1, 2] 領: [1, 0, 0, 0, 0, 1, 0, 2] 袖: [1, 0, 0, 0, 0, 1, 3] 首: [1, 0, 0, 0, 0, 2, 2] 𠂔: [1, 0, 0, 0, 1, 1, 2] 挨: [1, 0, 0, 0, 2, 0, 2] 撈: [1, 0, 0, 0, 2, 3] 彩: [1, 0, 0, 0, 3, 2] 使: [1, 0, 0, 1, 0, 0, 0, 2] 同: [1, 0, 0, 1, 0, 0, 3] 己: [1, 0, 0, 1, 0, 1, 2] 申: [1, 0, 0, 1, 1, 0, 2] 聞: [1, 0, 0, 1, 1, 3] 𠂔: [1, 0, 0, 1, 2, 2] 付: [1, 0, 0, 2, 0, 0, 2] 泊: [1, 0, 0, 2, 0, 3] 業: [1, 0, 0, 2, 1, 2] 誇: [1, 0, 0, 3, 0, 2] 回: [1, 0, 0, 3, 3] 短: [1, 0, 1, 0, 0, 0, 0, 2] 敬: [1, 0, 1, 0, 0, 0, 3] 調: [1, 0, 1, 0, 0, 1, 2] 相: [1, 0, 1, 0, 1, 0, 2] 𠂔: [1, 0, 1, 0, 1, 3] 貶: [1, 0, 1, 0, 2, 2] 疑: [1, 0, 1, 1, 0, 0, 2] 問: [1, 0, 1, 1, 0, 3] ヤ: [1, 0, 1, 1, 1, 2] 礼: [1, 0, 1, 2, 3] 𠂔: [1, 0, 2, 0, 0, 3] 演: [1, 0, 2, 1, 3] ニ: [1, 0, 3, 0, 3] 優: [1, 1, 0, 0,

0, 0, 3] 秀: [1, 1, 0, 0, 1, 3] 𠄎: [1, 1, 0, 1, 0, 3] 管: [1, 1, 0, 2, 3] 理: [1, 1, 1, 0, 0, 3] 𠄎: [1, 1, 1, 1, 3] 受: [1, 1, 2, 0, 3] 肩: [1, 1, 3, 3] 態: [1, 2, 0, 0, 0, 3] 侮: [1, 2, 0, 1, 3] 蔑: [1, 2, 1, 0, 3] 万: [1, 2, 2, 3] 逆: [1, 3, 0, 0, 3] 充: [1, 3, 1, 3] 認: [2, 0, 0, 0, 0, 3] 識: [2, 0, 0, 0, 1, 3] 恭: [2, 0, 0, 1, 0, 3] 揖: [2, 0, 0, 2, 3] 丁: [2, 0, 1, 0, 0, 3] 寧: [2, 0, 1, 1, 3] 謙: [2, 0, 2, 0, 3] 虚: [2, 0, 3, 3] 惜: [2, 1, 0, 0, 0, 3] 護: [2, 1, 0, 1, 3] 務: [2, 1, 1, 0, 3] 料: [2, 1, 2, 3] 機: [2, 2, 0, 0, 3] 嫌: [2, 2, 1, 3] 𠄎: [2, 3, 0, 3] 摘: [3, 0, 0, 0, 0, 3] 折: [3, 0, 0, 1, 3] 快: [3, 0, 1, 0, 3] 褒: [3, 0, 2, 3] 逃: [3, 1, 0, 0, 3] 𠄎: [3, 1, 1, 3] 念: [3, 2, 0, 3] 傘: [3, 3, 3] 独: [0, 0, 0, 2, 0, 0, 0, 3] 𠄎: [0, 0, 0, 2, 0, 1, 3] 𠄎: [0, 0, 0, 2, 1, 0, 3] 𠄎: [0, 0, 0, 2, 2, 3] 冷: [0, 0, 0, 3, 0, 0, 3] 漏: [0, 0, 0, 3, 1, 3] 雄: [0, 0, 1, 2, 0, 0, 3] 弁: [0, 0, 1, 2, 1, 3] 案: [0, 0, 1, 3, 0, 3] 千: [0, 1, 0, 1, 0, 0, 0, 3] 百: [0, 1, 0, 1, 0, 1, 3] 五: [0, 1, 0, 1, 1, 0, 3] 𠄎: [0, 1, 0, 1, 2, 3] 𠄎: [0, 1, 0, 2, 0, 0, 3] 𠄎: [0, 1, 0, 2, 1, 3] 尋: [0, 1, 0, 3, 0, 3] 導: [0, 1, 1, 0, 0, 0, 0, 3] 夫: [0, 1, 1, 0, 0, 1, 3] 答: [0, 1, 1, 0, 1, 0, 3] 眉: [0, 1, 1, 0, 2, 3] 毛: [0, 1, 1, 1, 0, 0, 3] 崇: [0, 1, 1, 1, 1, 3] 高: [0, 1, 1, 2, 0, 3] 簡: [0, 1, 1, 3, 3] ケ: [0, 1, 2, 0, 0, 0, 3] 吟: [0, 1, 2, 0, 1, 3] 𠄎: [0, 1, 2, 1, 0, 3] 𠄎: [0, 1, 2, 2, 3] 岩: [0, 2, 0, 0, 0, 0, 0, 3] 頑: [0, 2, 0, 0, 0, 1, 3] 架: [0, 2, 0, 0, 1, 0, 3] 構: [0, 2, 0, 0, 2, 3] 莫: [0, 2, 0, 1, 0, 0, 3] 抱: [0, 2, 0, 1, 1, 3] 𠄎: [0, 2, 0, 2, 0, 3] 裂: [0, 2, 0, 3, 3] 煉: [0, 2, 2, 0, 0, 3] 瓦: [0, 2, 2, 1, 3] 𠄎: [0, 2, 3, 0, 3] 𠄎: [0, 3, 0, 0, 0, 0, 3] 筋: [0, 3, 0, 0, 1, 3] 𠄎: [0, 3, 0, 1, 0, 3] 割: [0, 3, 0, 2, 3] 𠄎: [0, 3, 2, 0, 3] 妻: [0, 3, 3, 3] 刻: [1, 0, 0, 0, 0, 0, 0, 3] 印: [1, 0, 0, 0, 0, 1, 3] 諺: [1, 0, 0, 0, 0, 1, 0, 3] 半: [1, 0, 0, 0, 0, 2, 3] 𠄎: [1, 0, 0, 0, 1, 1, 3] 形: [1, 0, 0, 0, 2, 0, 3] 背: [1, 0, 0, 0, 3, 3] 載: [1, 0, 0, 1, 0, 0, 0, 3] ! : [1, 0, 0, 1, 0, 1, 3] 胆: [1, 0, 0, 1, 1, 0, 3] 連: [1, 0, 0, 1, 2, 3] 𠄎: [1, 0, 0, 2, 0, 0, 3] ゼ: [1, 0, 0, 2, 1, 3] 定: [1, 0, 0, 3, 0, 3] 𠄎: [1, 0, 1, 0, 0, 0, 0, 3] 天: [1, 0, 1, 0, 0, 1, 3] 登: [1, 0, 1, 0, 1, 0, 3] ギ: [1, 0, 1, 0, 2, 3] べ: [1, 0, 1, 1, 0, 0, 3] 拭: [1, 0, 1, 1, 1, 3]

5 Quellcode

5.1 TreeNode Klasse

```

1  class TreeNode:
2
3      def __init__(self, cost, parent, perln):
4          self.perln = perln
5          self.cost = cost
6          self.children = []
7          self.parent = parent
8          self.is_leaf = True
9          self.is_protected = False # Wenn True, dann kann er keine kinder mehr bekommen, weil ihm bei einer im
10
11     def insert_new_node(self, obj):
12         if obj is self:
13             if self.is_leaf: # Damit jede parent immer mehr mindestens zwei kinder hat
14                 self.is_leaf = False
15                 self.children.append(TreeNode(self.cost + perl_costs[0], self, self.perln+[len(self.children)]))
16
17                 self.children.append(TreeNode(self.cost + perl_costs[len(self.children)], self, self.perln+[len(self.children)]))
18                 return
19
20         for child in self.children:
21             child.insert_new_node(obj)
22
23     def get_size(self) -> int:
24         if self.is_leaf:
25             return 1
26         return sum(child.get_size() for child in self.children)
27
28     def get_leaves(self) -> list:
29         if self.is_leaf:
30             return [self]
31
32         a = []
33         for child in self.children:
34             for i in child.get_leaves():
35                 a.append(i)
36         return a
37
38     def get_all_possible_costs(self) -> dict:
39         a = {}
40         if len(self.children) < len(perl_costs) and self.is_protected is False:
41             if self.is_leaf:
42                 a[self] = self.cost + perl_costs[0] + self.cost + perl_costs[1]
43             else:

```

```

        a[self] = self.cost + perl_costs[len(self.children)]
45
        for child in self.children:
47             a.update(child.get_all_possible_costs())
        return a
49
    def get_interal_nodes(self):
51         if self.is_leaf:
            return 0
53         return sum(child.get_interal_nodes() for child in self.children) + 1

```

5.2 main.py

```

1  def create_distribution(message: str):
    characters = {}
3     for char in message:
        if char in characters.keys():
5         characters[char] += 1
        else:
7         characters[char] = 1
    characters = dict(sorted(characters.items(), key=lambda item: item[1], reverse=True))
9     return [key for key in characters.keys()], list(characters.values())

11
12  def create_tree(root: TreeNode) -> TreeNode:
13     while root.get_size() < len(distribution):
        possible_children = root.get_all_possible_costs()
15         smallest_value = min(possible_children.items(), key=lambda item: item[1])

17         root.insert_new_node(smallest_value[0])
    return root
19

21  def calculate_tree(root):
    all_leaves = root.get_leaves()
23     all_leaves = sorted(all_leaves, key=lambda x: (x.cost, len(x.parent.children)))
    total_count = 0
25     for i in range(len(all_leaves)):
        total_count += all_leaves[i].cost * distribution[i]
27     return total_count

29
30  def improve_tree(root: TreeNode) -> TreeNode:
31     all_leaves = root.get_leaves()
    all_leaves = sorted(all_leaves, key=lambda x: (x.cost, len(x.parent.children)))
33     a = 0
    b = None
35     for i in range(len(all_leaves)):
        if all_leaves[i].parent.parent is not None:
37             if all_leaves[i].cost * distribution[i] > a:
                a = all_leaves[i].cost * distribution[i]
39                 b = all_leaves[i].parent

41     b.is_leaf = True
    b.children = []
43     b.is_protected = True

45     return create_tree(root)

47
48  def find_best_tree(root):
    best_value = calculate_tree(root)
    best_root = copy.deepcopy(root)
51     while True:
        try:
53         root = improve_tree(root)
        new_calc = calculate_tree(root)
55         if new_calc < best_value:
            best_value = new_calc
            best_root = copy.deepcopy(root)
57

```

```
        except Exception as e:
59             return best_root

61
62 def main():
63     root = TreeNode(0, None, [])
64     root = create_tree(root)
65     root = find_best_tree(root)
```

6 Quellen

- [1] Mordecai J. Golin and Günther Rote, *A Dynamic Programming Algorithm for Constructing Optimal Prefix-Free Codes with Unequal Letter Costs*, IEEE Transactions on Information Theory 44, no. 5, (September 1998)
- [2] Mordecai J. Golin, Claire Mathieu und Neal E. Young, *HUFFMAN CODING WITH LETTER COSTS: A LINEAR-TIME APPROXIMATION SCHEME*, 2012 Society for Industrial and Applied Mathematics Vol. 41, No. 3, pp. 684–713
- [3] Prof. Justus Piater, *Algorithmen und Datenstrukturen*, 29.05.2024 Universität Innsbruck Seite 4-13